

CS498JH: Introduction to NLP (Fall 2012)

<http://cs.illinois.edu/class/cs498jh>

Lecture 16:

Feature structures

and unification

Julia Hockenmaier

juliahmr@illinois.edu

3324 Siebel Center

Office Hours: Wednesday, 12:15-1:15pm

Today's lecture

Feature structures form the basis for many grammar formalisms used in computational linguistics.

Feature structure grammars (aka **attribute-value grammars**, or **unification grammars**) can be used as

- a more compact way of representing rich CFGs
- a way to represent more expressive grammars

Simple grammars overgenerate

$$\begin{aligned} S &\rightarrow NP \ VP \\ VP &\rightarrow Verb \ NP \\ NP &\rightarrow Det \ Noun \\ Det &\rightarrow the \mid a \mid these \\ Verb &\rightarrow eat \mid eats \\ Noun &\rightarrow cake \mid cakes \mid student \mid students \end{aligned}$$

This generates ungrammatical sentences like
“*these student eats a cakes*”

We need to capture (number/person) agreement

Features as atomic objects

$$S \rightarrow NP_{sg} VP_{sg}$$
$$S \rightarrow NP_{pl} VP_{pl}$$
$$VP_{sg} \rightarrow VerbSg NP$$
$$VP_{pl} \rightarrow VerbPl NP$$
$$NP_{sg} \rightarrow DetSg NounSg$$
$$DetSg \rightarrow the \mid a$$

... ..

This yields **very large grammars**.

(What about person, case, ...?)

Difficult to capture **generalizations**.

(Subject and verb have to have number agreement)

Feature structures

Replace atomic categories with feature structures:

CAT	NP
NUM	SG
PERS	3
CASE	NOM

CAT	VP
NUM	SG
PERS	3
VFORM	FINITE

A **feature structure** is a list of **features** (= attributes), e.g. CASE, and **values** (eg NOM).

We often represent feature structures as **attribute value matrices (AVM)**

Usually, values are **typed** (to avoid CASE:SG)

Complex feature structures

We distinguish between atomic and complex feature values.

A complex value is a feature structure itself.

This allows us to capture better generalizations.

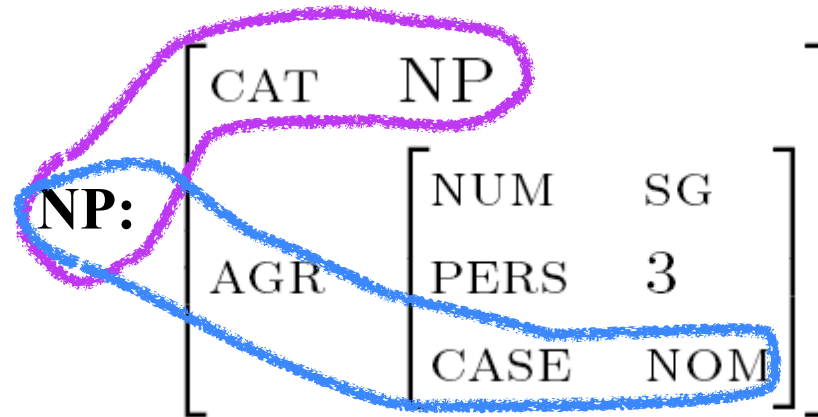
Only atomic values:

CAT	NP
NUM	SG
PERS	3
CASE	NOM

Complex values:

CAT	NP						
AGR	<table><tr><td>NUM</td><td>SG</td></tr><tr><td>PERS</td><td>3</td></tr><tr><td>CASE</td><td>NOM</td></tr></table>	NUM	SG	PERS	3	CASE	NOM
NUM	SG						
PERS	3						
CASE	NOM						

Feature paths



A **feature path** allows us to identify particular values in a feature structure:

$\langle \text{NP CAT} \rangle = \text{NP}$

$\langle \text{NP AGR CASE} \rangle = \text{NOM}$

A grammar with feature structures

S	→ NP VP	Grammar rule
$\langle \mathbf{NP} \text{ NUM} \rangle$	$= \langle \mathbf{VP} \text{ NUM} \rangle$	Constraints
$\langle \mathbf{NP} \text{ CASE} \rangle$	$= \textit{nom}$	
NP	→ DT NOUN	Grammar rule
$\langle \mathbf{NP} \text{ NUM} \rangle$	$= \langle \mathbf{NOUN} \text{ NUM} \rangle$	Constraints
$\langle \mathbf{NP} \text{ CASE} \rangle$	$= \langle \mathbf{NOUN} \text{ CASE} \rangle$	
NOUN	→ <i>cake</i>	Lexical entry
$\langle \mathbf{NOUN} \text{ NUM} \rangle$	$= \textit{sg}$	Constraints

PATR-II style feature structures

CFG rules are augmented with constraints:

$$\mathbf{A}_0 \rightarrow \mathbf{A}_1 \dots \mathbf{A}_n$$
$$\{\text{set of constraints}\}$$

There are two kinds of constraints:

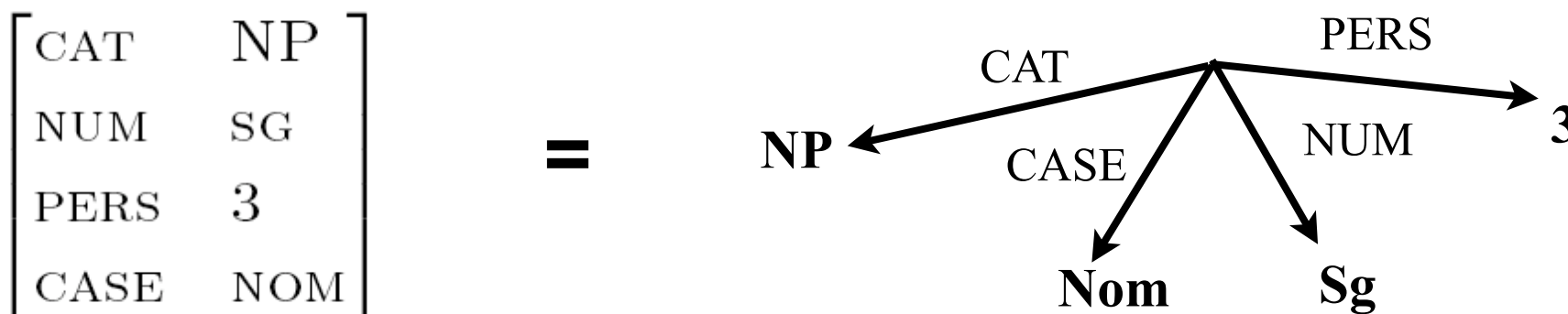
Unification constraints:

$$\langle \mathbf{A}_i \text{ feature-path} \rangle = \langle \mathbf{A}_j \text{ feature-path} \rangle$$

Value constraints:

$$\langle \mathbf{A}_i \text{ feature-path} \rangle = \text{atomic value}$$

Feature structures as directed graphs



Unification

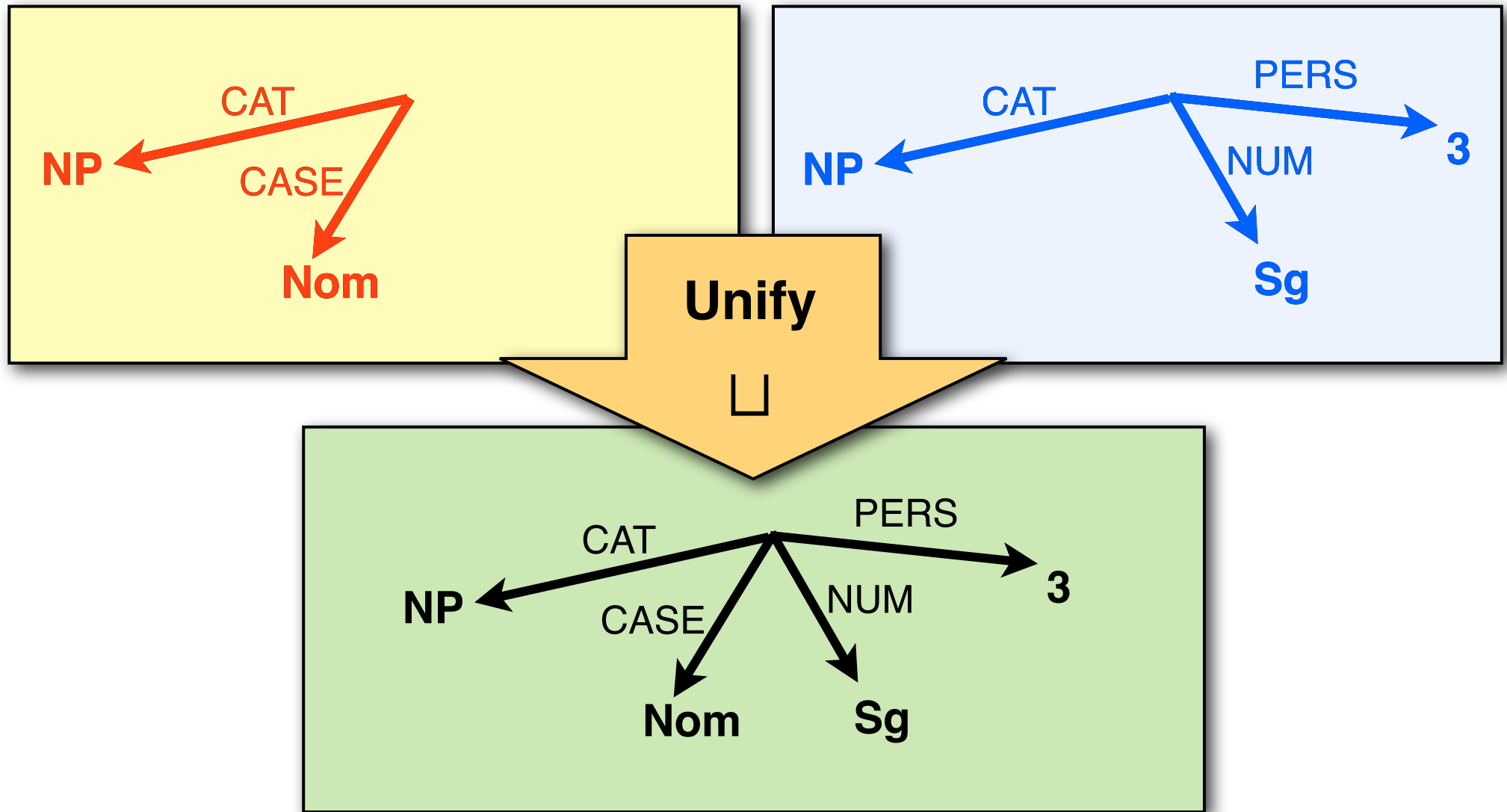
Two **feature structures A and B unify** ($A \sqcup B$) if they can be merged into one consistent feature structure C:

$$\begin{bmatrix} \text{CAT} & \text{NP} \\ \text{NUM} & \text{SG} \\ \text{CASE} & \text{NOM} \end{bmatrix} \sqcup \begin{bmatrix} \text{CAT} & \text{NP} \\ \text{PERS} & 3 \end{bmatrix} = \begin{bmatrix} \text{CAT} & \text{NP} \\ \text{NUM} & \text{SG} \\ \text{PERS} & 3 \\ \text{CASE} & \text{NOM} \end{bmatrix}$$

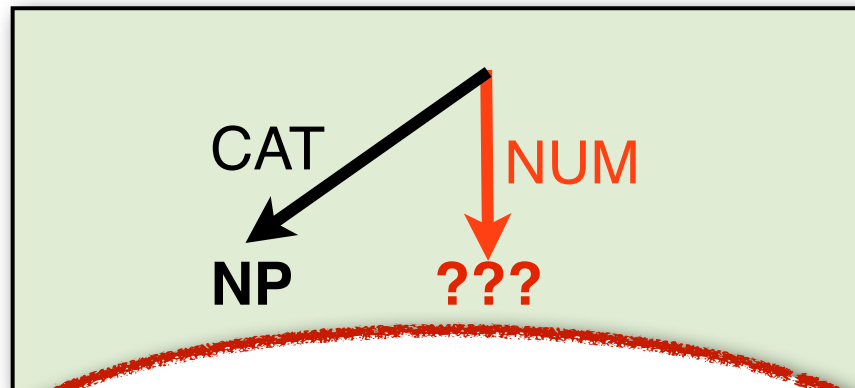
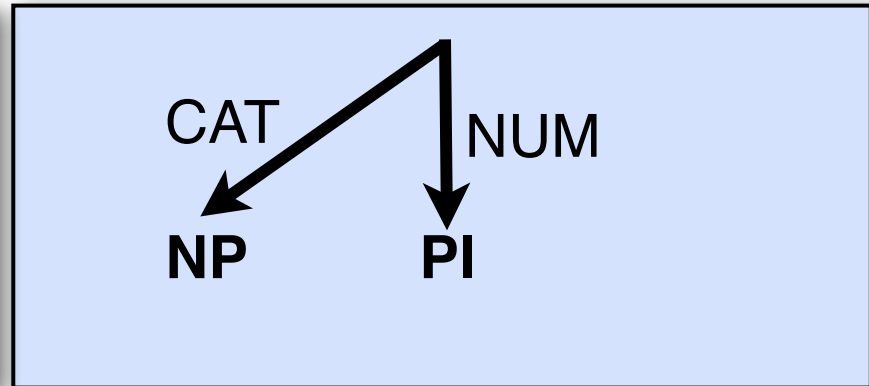
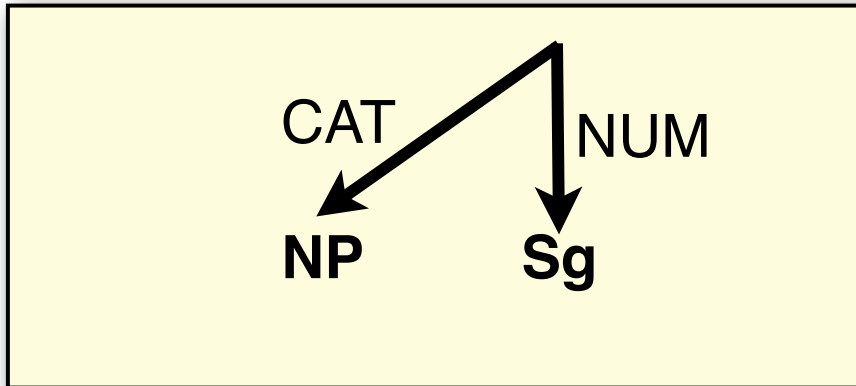
Otherwise, unification **fails**:

$$\begin{bmatrix} \text{CAT} & \text{NP} \\ \text{NUM} & \text{SG} \\ \text{CASE} & \text{NOM} \end{bmatrix} \sqcup \begin{bmatrix} \text{CAT} & \text{NP} \\ \text{NUM} & \text{PL} \end{bmatrix} = \emptyset$$

Unification as graph-matching



Unification as graph-matching



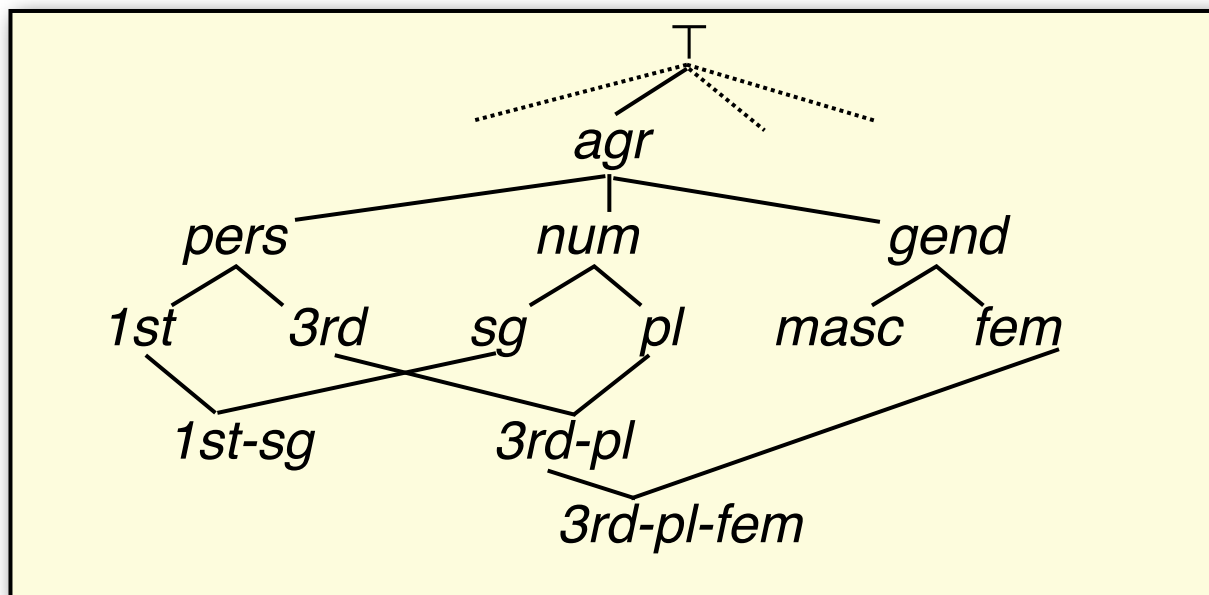
Unification failure !

Typed feature structures

In a **typed feature structure** system,

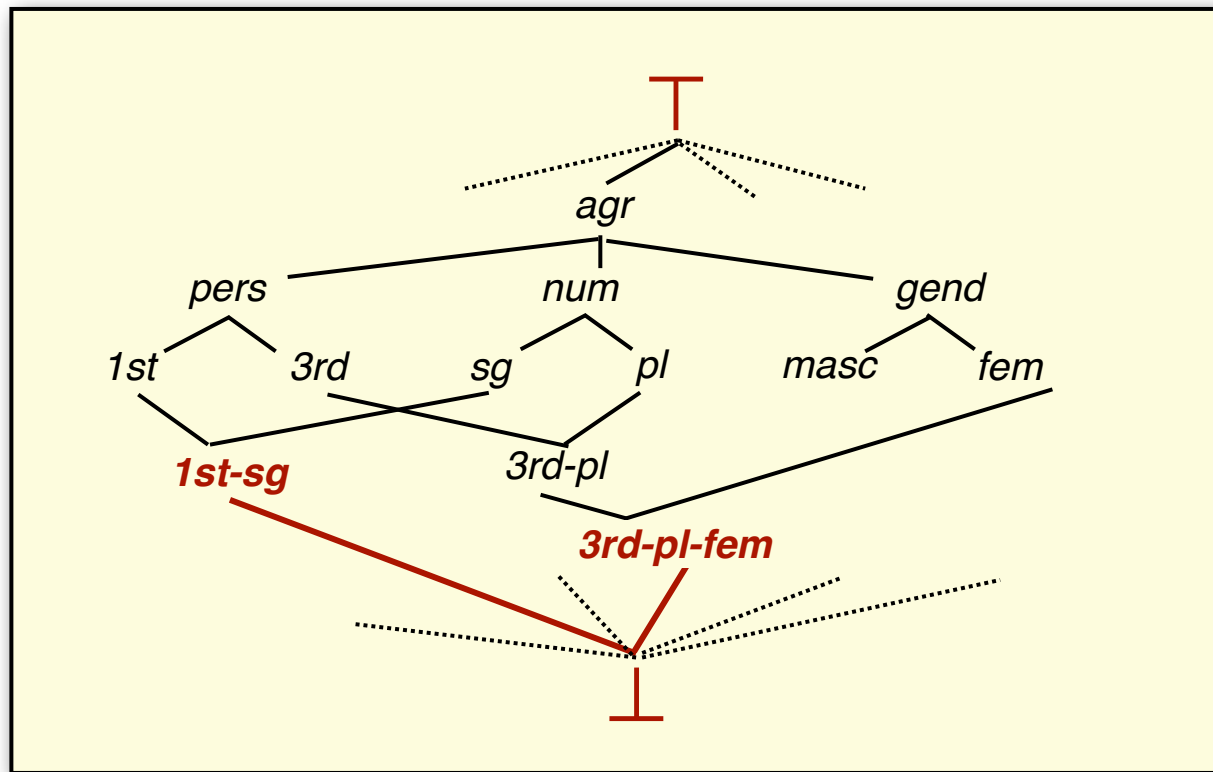
- each feature structure has a **type**
- each **type specifies which attributes** it has
- the **values of each attribute are typed**
- types are arranged in a **multiple inheritance hierarchy**:

$\top$ ('top') is the root, *pers* is a subtype of *agr*, *3rd-pl-fem* is a subtype of *3rd-pl* and *fem* (and of *3rd*, *gend*, *pl*, *pers*, ..., *agr*)

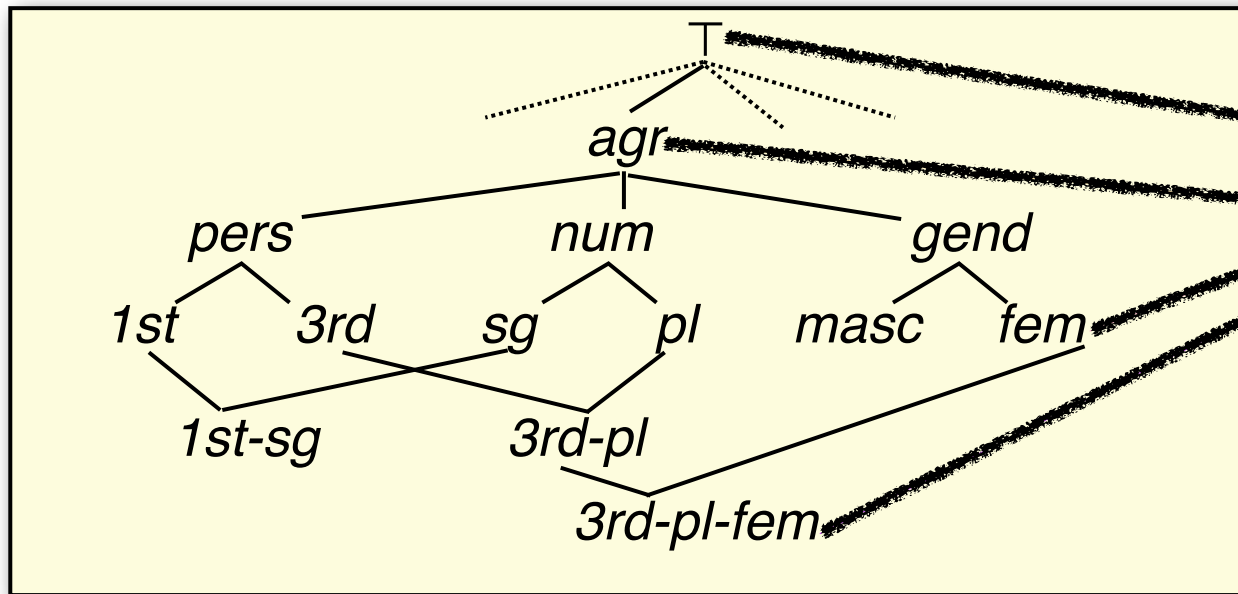


Another view of unification failure

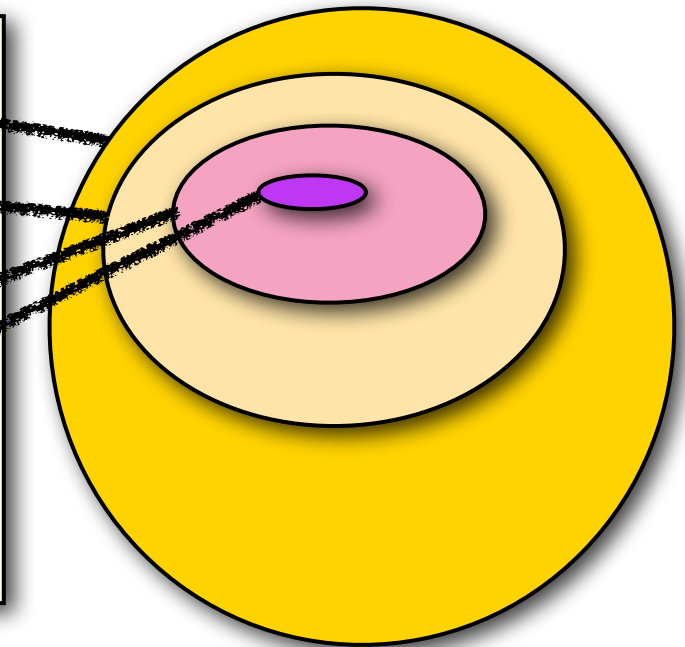
- $\top$ ('top') is the **root** of every type hierarchy
(= the most general supertype)
- $\perp$ ('bottom') is a **subtype** of every type.



Features as constraints



Type Hierarchy



**Universe of
'linguistic objects'
(=strings)**

Features as constraints

Features impose **constraints** on linguistic objects.

If A and B unify, but B contains more features than A, B is more specific than A:

$\begin{bmatrix} \text{CAT} & \text{NP} \\ \text{NUM} & \text{SG} \\ \text{CASE} & \text{NOM} \end{bmatrix}$	$\begin{bmatrix} \text{CAT} & \text{NP} \\ \text{NUM} & \text{SG} \\ \text{PERS} & 3 \\ \text{CASE} & \text{NOM} \end{bmatrix}$
A	B

We also say that **A subsumes B**.

Subsumption defines a *partial ordering* over feature structures.

The head feature

Instead of implicitly specifying heads for each rewrite rule, let us define a **head feature**.

The head of a VP has the same agreement feature as the VP itself:

$$\left[\begin{array}{c} \text{CAT} \\ \text{AGR} \\ \text{HEAD} \end{array} \begin{array}{c} \text{VP} \\ \left[\begin{array}{cc} \text{NUM} & \text{SG} \\ \text{PERS} & 3 \end{array} \right] \\ \left[\begin{array}{c} \text{AGR} \\ \left[\begin{array}{cc} \text{NUM} & \text{SG} \\ \text{PERS} & 3 \end{array} \right] \end{array} \right] \end{array} \right]$$

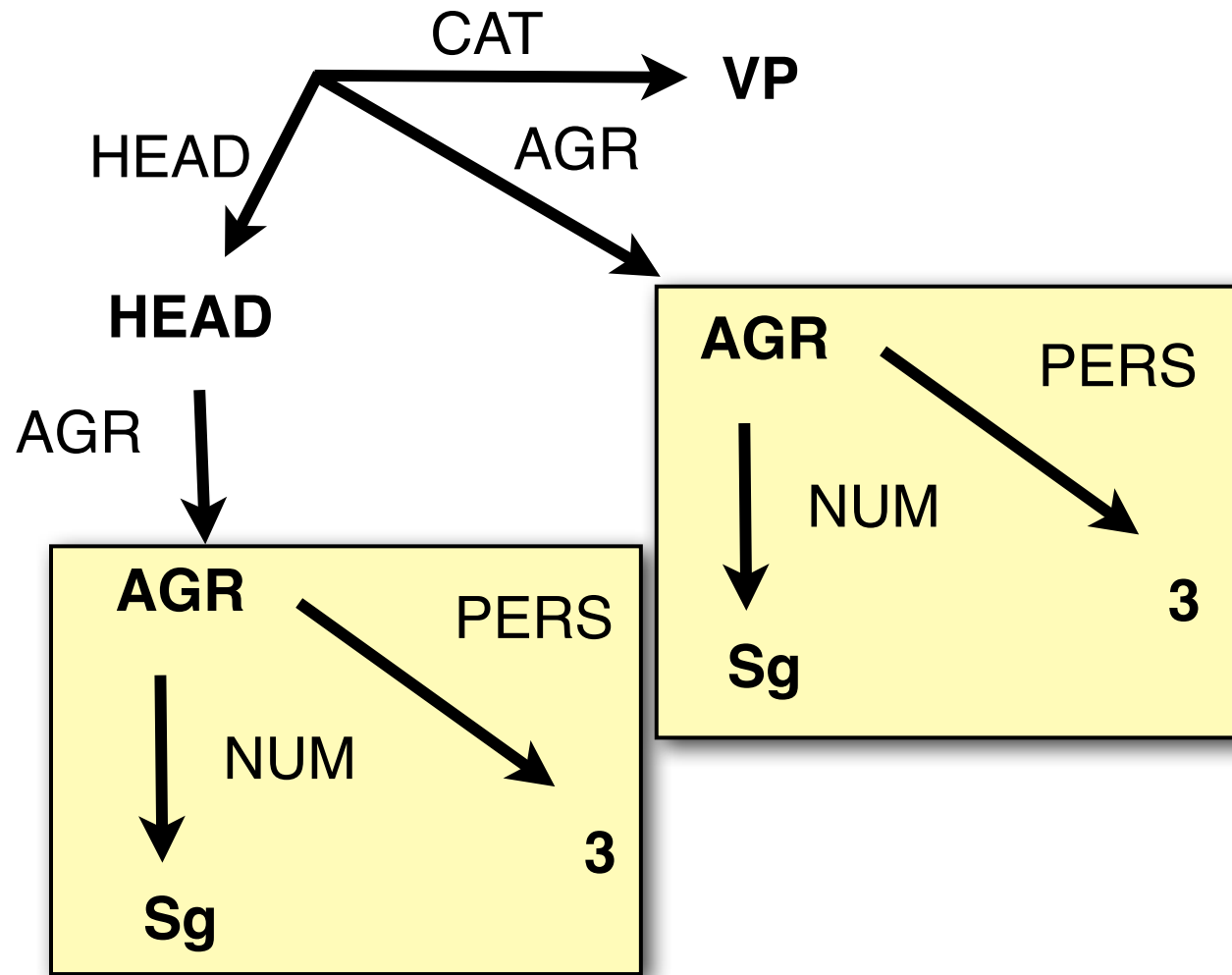
Re-entrancies

What we *really* want to say is that the agreement feature of the head is *identical* to that of the VP itself.

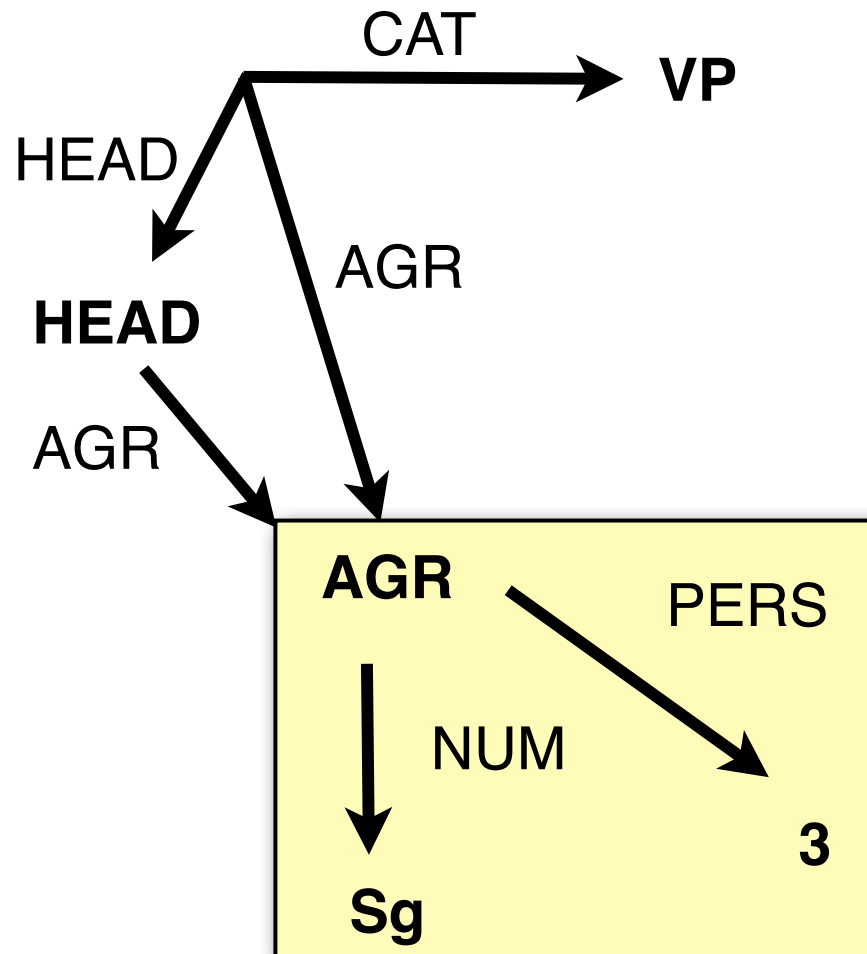
This corresponds to a **re-entrancy** in the FS:

$$\left[\begin{array}{cc} \text{CAT} & \text{VP} \\ \text{AGR} & \boxed{1} \left[\begin{array}{cc} \text{NUM} & \text{SG} \\ \text{PERS} & 3 \end{array} \right] \\ \text{HEAD} & \left[\text{AGR} \boxed{1} \right] \end{array} \right]$$

Re-entrancies - not like this:



Re-entrancies - but like this:



Extensions of feature structures

Disjunction:

eats: [PERS 3], *eat*: [PERS: $1 \vee 2$]

Negation:

eats: [PERS 3] *eat*: [PERS: $\neg 3$]

List-valued features:

English *give* takes an NP and a to-PP as arguments, and they have to appear in a specific order:

“give the book to you” not: **“give to you the book”*

give: [CAT: VP, SUBCAT: $\langle \text{NP}, \text{PP}_{\text{to}} \rangle$]

Set-valued features:

German *geben* takes three NPs, which can appear in any order:

ich gebe dir das Buch, das Buch gebe ich dir, dir gebe ich das Buch,...

geben: [CAT: S, SUBCAT {NP_{nom} NP_{acc}, NP_{dat}}]

Attribute-Value Grammars and CFGs

If every feature can only have **a finite set of values**, any attribute-value grammar can be compiled out into a (possibly huge) context-free grammar

Going beyond CFGs

The power-of-2 language: $L_2 = \{w^i \mid i \text{ is a power of } 2\}$

L_2 is a (fully) context-sensitive language.

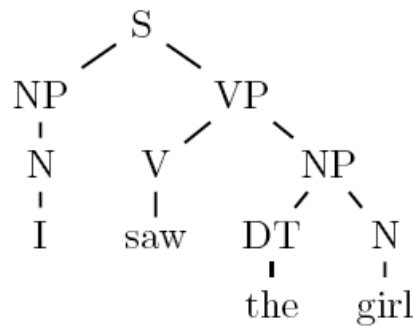
(*Mildly* context-sensitive languages have the **constant growth property** (the length of words always increases by a constant factor c))

Here is a feature grammar which generates L_2 :

$$\begin{array}{lcl} A & \rightarrow & a \\ & & [F : 1] \\ A & \rightarrow & A' A'' \\ & & \langle AF = A' \rangle \ \& \ \langle AF = A'' \rangle \end{array}$$

Supplementary material: Unification-based formalisms

Lexical Functional Grammar (LFG)



SUBJ	[	PRED	'PRO'
		PERS	1
		NUM	SG
]			
TENSE	PAST		
PRED	'SEE<((↑ SUBJ), (↑ OBJ))>'		
OBJ	[	PRED	'GIRL'
		DEF	+
		PERS	3
		NUM	SG
]			

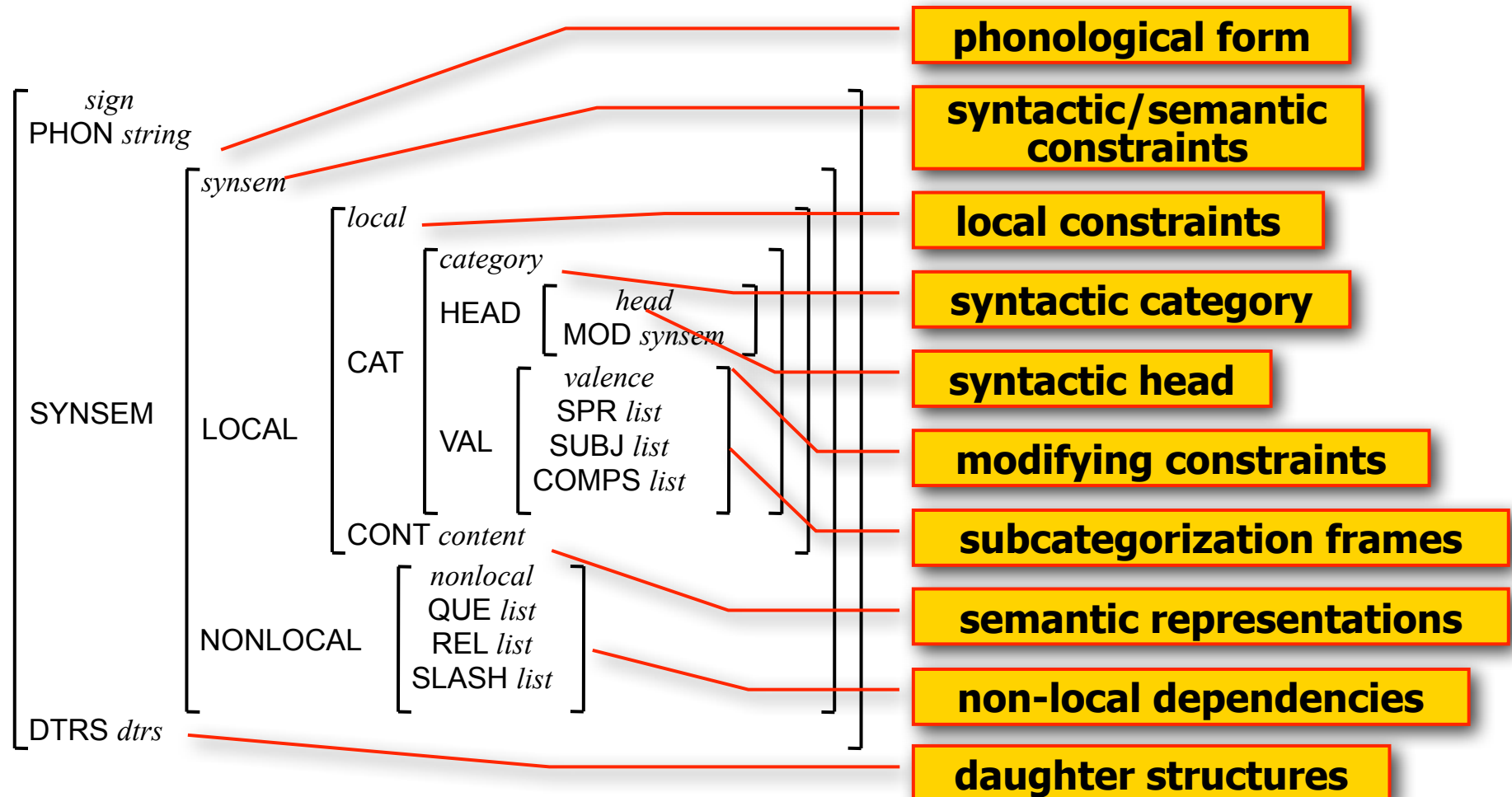
Uses **phrase structure trees** to represent constituent structure (**c-structure**)

- C-structure describes how words and phrases are arranged to form sentences in a particular language.

Uses **feature structures** (attribute-value matrices) to represent “functional structure” (**f-structure**)

- F-structure defines language-universal grammatical functions (subject, object) and abstract features (case, tense).

HPSG signs (feature structures)



Today's key concepts

Feature structure grammars

- feature structures: AVMs and DAGs
- unification and unification failure
- subsumption
- reentrancies